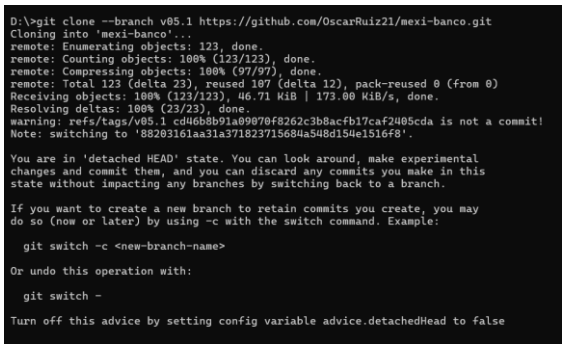
Javier Velasco Pacheco
S05

MEXI-BANCO

El objetivo principal de esta práctica es desplegar y probar localmente la aplicación de "Mexi Banco" utilizando Docker Compose, con la finalidad de comprender su estructura.

Paso 1. Descarga de Repo

Primero debemos tener descargada la aplicación de escritorio de Postman, descargar la colección correspondiente, clonar el repositorio mediante el comando git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git



Paso 2 · Docker Compose

Una vez clonado el repositorio, procederemos a levantar el entorno con el comando docker compose up --build -d, comprobando que esté correctamente operativo con docker compose ps, sabiendo que se levantó de forma exitosa porque su estado será *healthy*.



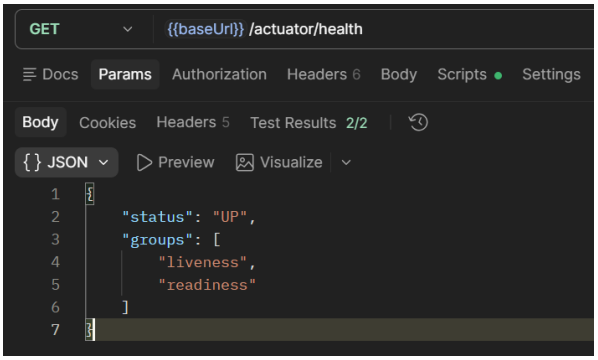


Paso 3 · Pruebas con Postman

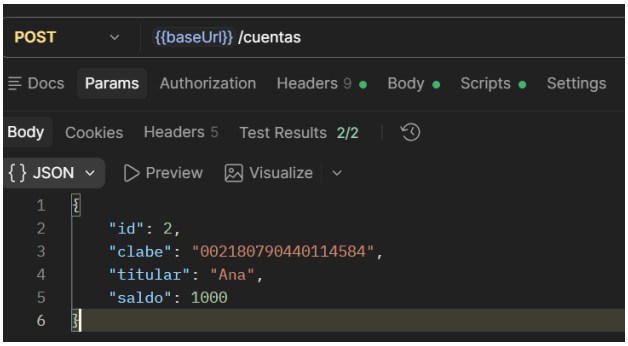
Para validar el funcionamiento del sistema, el siguiente paso consiste en verificar las rutas y operaciones de la API mediante Postman, utilizando para ello la colección previamente obtenida para ejecutar de manera secuencial cada uno de los *endpoints* disponibles.

- **Arriba y respondiendo.**

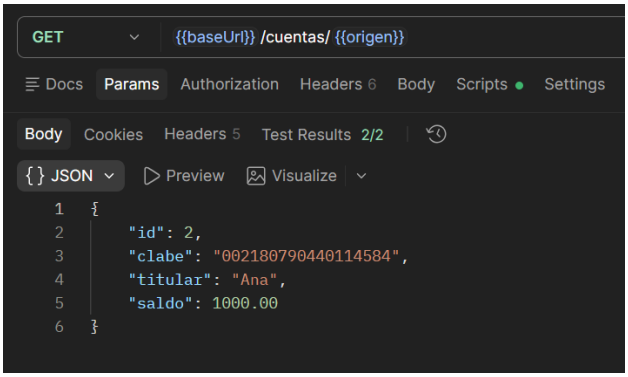
Probamos el estado del sistema.



Creamos una cuenta para Ana y le asiganmos un saldo.



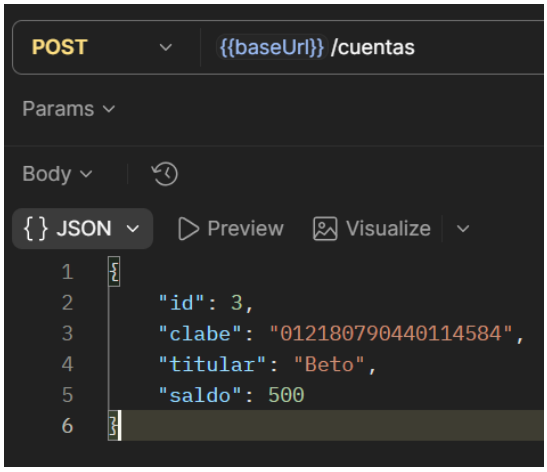
Consultaremos la cuenta que asignamos con GET.



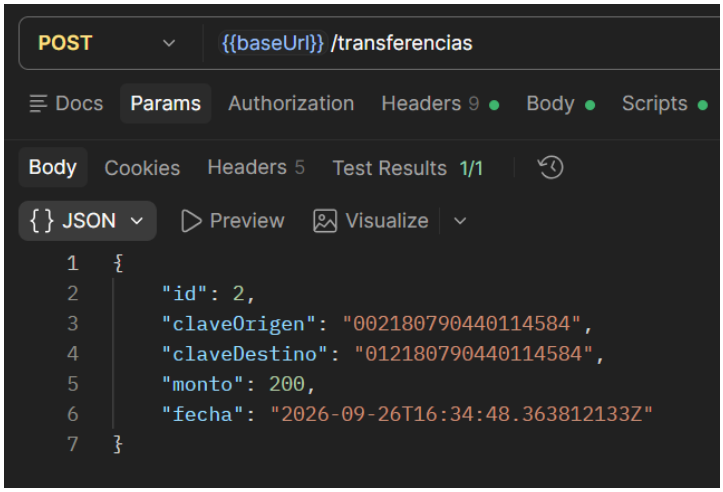
Primero se comprobó que la aplicación estuviera disponible de salud, obteniendo 200 OK y estado UP. Después se creó una cuenta con saldo inicial de 1000 y se consultó nuevamente para comprobar que quedó registrada correctamente.

- **Transferencia Interna**

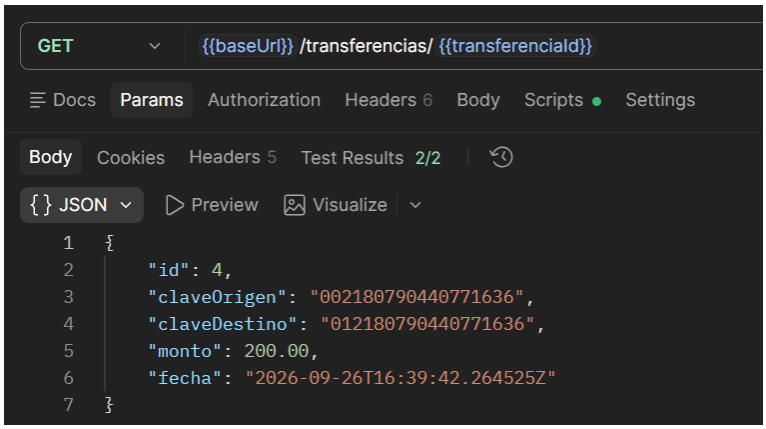
Definimos una cuenta de destino para la transferencia, Beto con saldo de 500



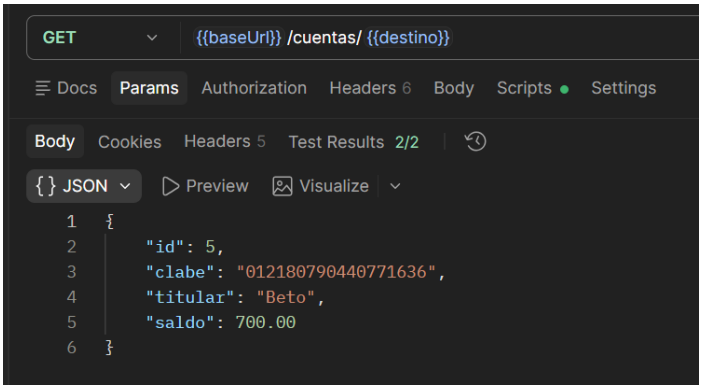
Se manda de Ana a Beto con un monto de 200.

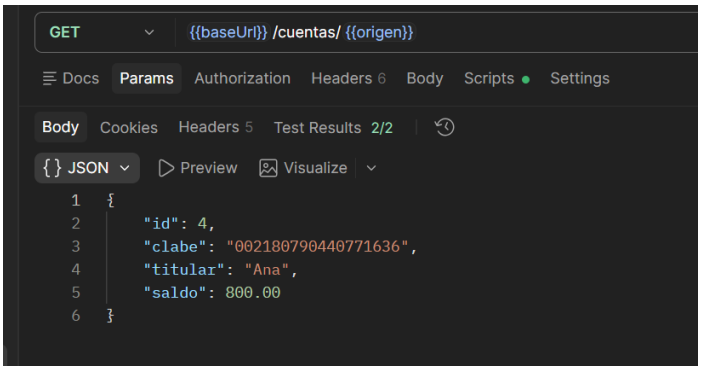


Consultamos la transferencia que se realizó en cuentas



Debido a la transacción, Ana termino con un saldo 800 y Beto con 700.



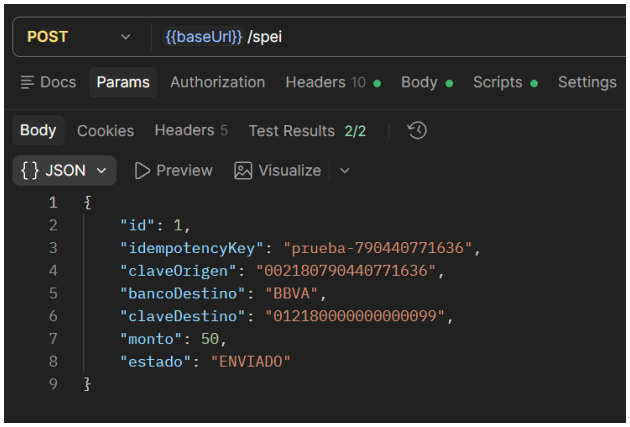


Se creó una segunda cuenta con saldo inicial de 500 y se realizó una transferencia de 200 desde la cuenta origen. Después de la operación, el saldo de la cuenta origen pasó de 1000 a 800, mientras que la cuenta destino pasó de 500 a 700. Finalmente, se consultó la transferencia para comprobar que quedó registrada correctamente.

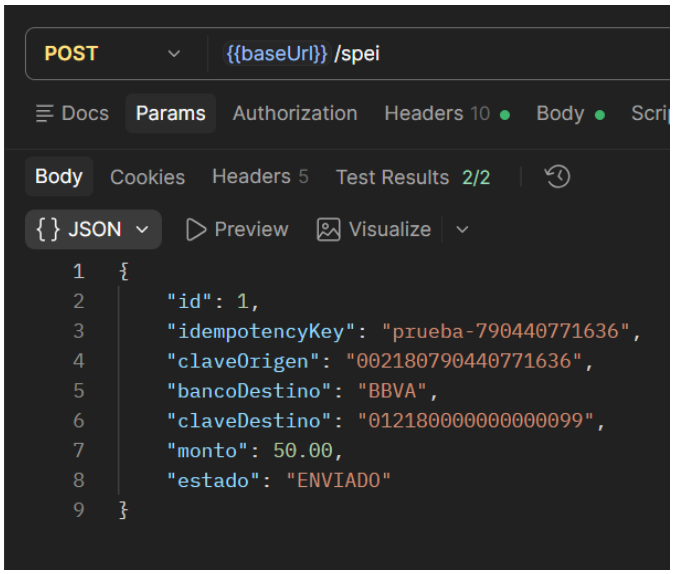
- **Idempotencia.**

Comprobaremos la propiedad de idempotencia. Este principio garantiza que ejecutar una misma operación varias veces genera el mismo resultado que hacerlo una sola vez, previniendo cargos duplicados en las cuentas.

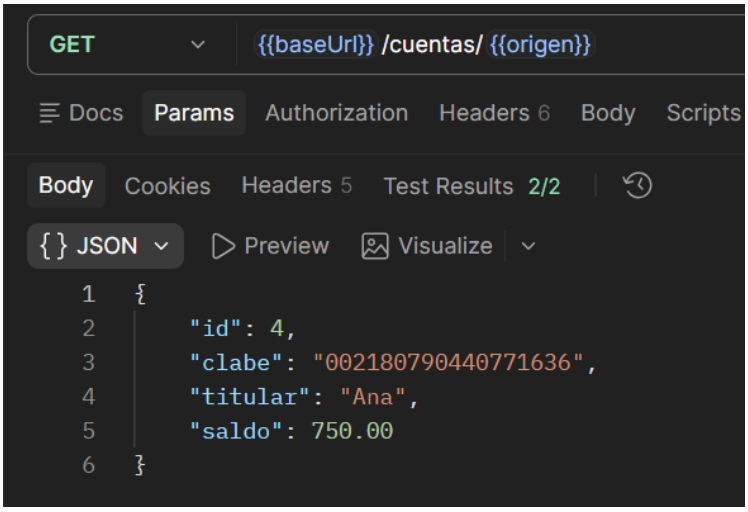
Ejecutaremos una transferencia inicial con el identificador único 1, descontando 50 unidades de la cuenta origen, cuyo saldo inicial era de 800, dejándola en un total de 750.



Posteriormente, simularemos un reenvío enviando la misma solicitud con exactamente el mismo identificador (id: 1).



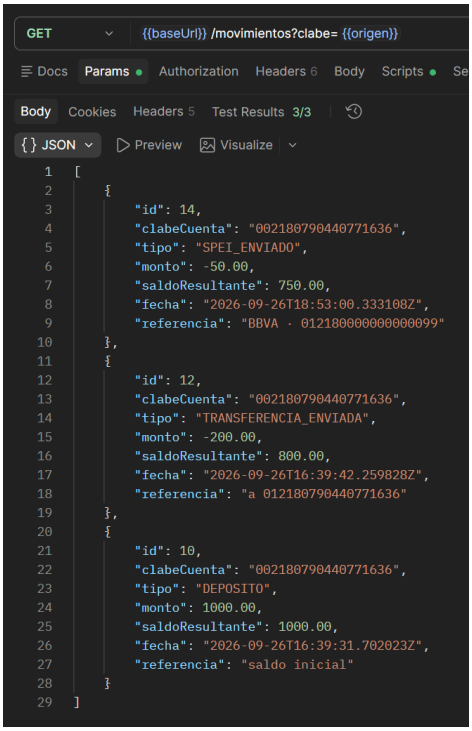
Al consultar nuevamente el estado de la cuenta origen, el saldo deberá permanecer intacto en 750, confirmando que el sistema reconoce la petición duplicada y evita procesarla de nuevo.



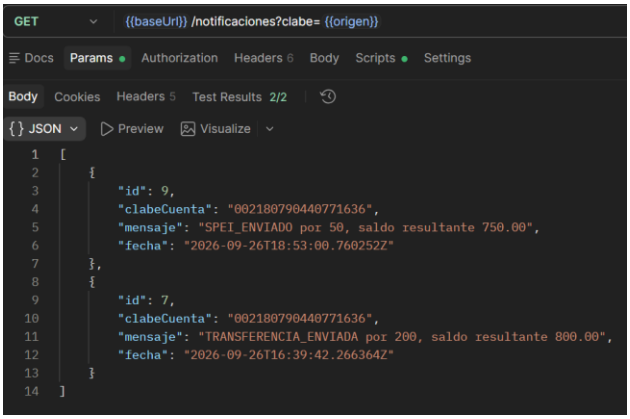
Se envió un SPEI de 50 y posteriormente se repitió la misma solicitud utilizando la misma Idempotency-Key. Ambas peticiones devolvieron el mismo identificador, por lo que no se creó una segunda operación. El saldo final fue de 750, comprobando que el monto solo se descontó una vez.

- **Monolito**

Podemos revisar todo los movimientos de las cuentas, tanto las transacciones y orígenes de cuenta.



Consultamos las notificaciones, donde están nuestros movimientos.



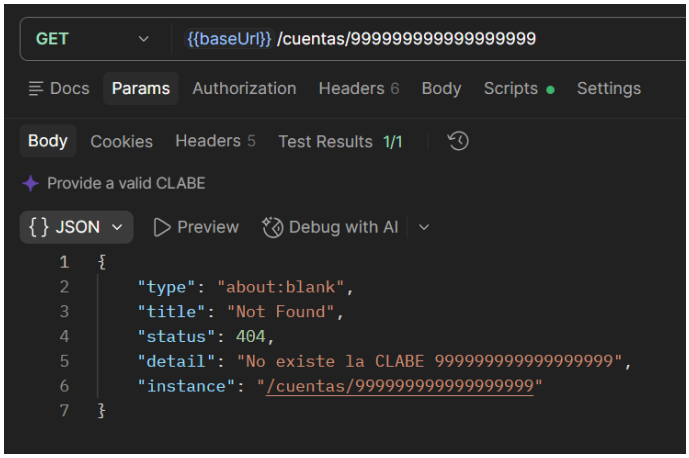
En esta parte se revisó la información que dejaron las operaciones realizadas anteriormente dentro del sistema. Al consultar los movimientos de la cuenta origen se comprobó que únicamente se registró un SPEI y que los movimientos corresponden con el saldo actual de 750. También se consultaron las notificaciones, obteniendo dos avisos: uno por la transferencia interna y otro por el SPEI.

- **Errores**

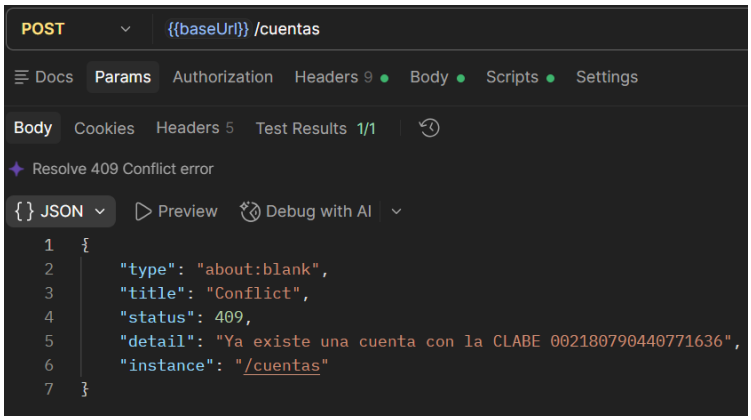
Se realizaron diversas pruebas con solicitudes inválidas para verificar que la aplicación manejara correctamente los errores y evitara realizar operaciones incorrectas. Se obtuvo una respuesta **404** al intentar acceder a una cuenta inexistente, mientras que el código **409** indicó que se intentó registrar una CLABE duplicada. Además, las solicitudes incompletas fueron rechazadas con el código **400**, y las transferencias no permitidas generaron una respuesta **422**.

También se comprobó que una operación SPEI enviada sin la cabecera Idempotency-Key fuera rechazada con el código **400**, ya que este identificador permite reconocer las solicitudes repetidas y evitar que una misma operación se procese más de una vez. Finalmente, se revisó el saldo de la cuenta después de realizar las pruebas y se confirmó que permaneció en **750**. Esto demuestra que las solicitudes rechazadas no modificaron el saldo ni produjeron cambios en los datos de la cuenta, lo que ayuda a mantener la integridad de la información y la confiabilidad del sistema.

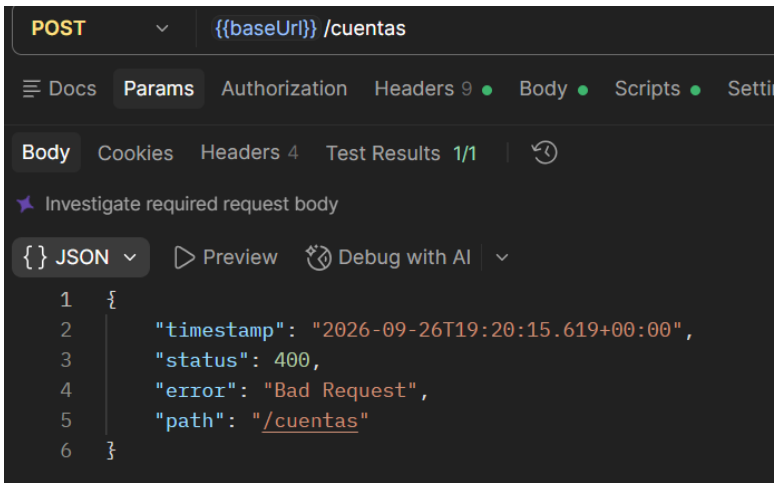
1. No es encontró la cuenta 99999...



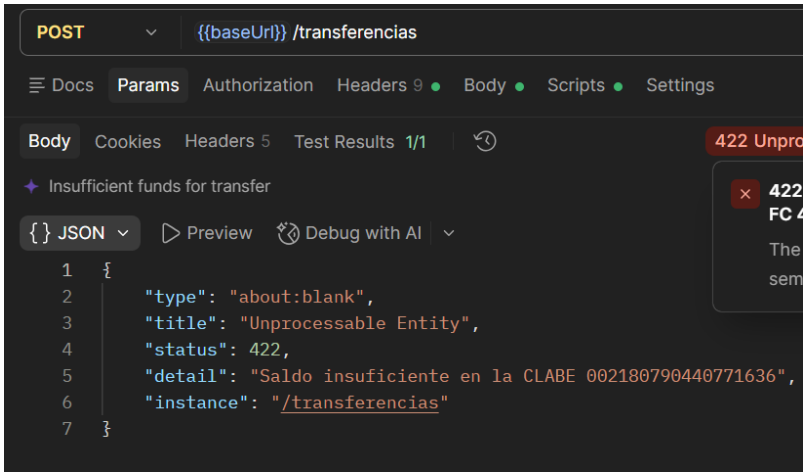
2. Cuando ya hay un clave existente



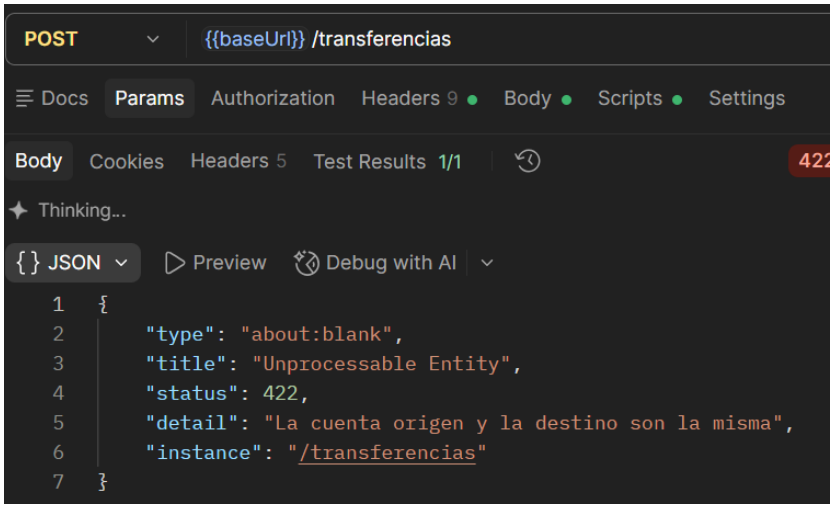
3. Cuando una solicitud sin cuerpo



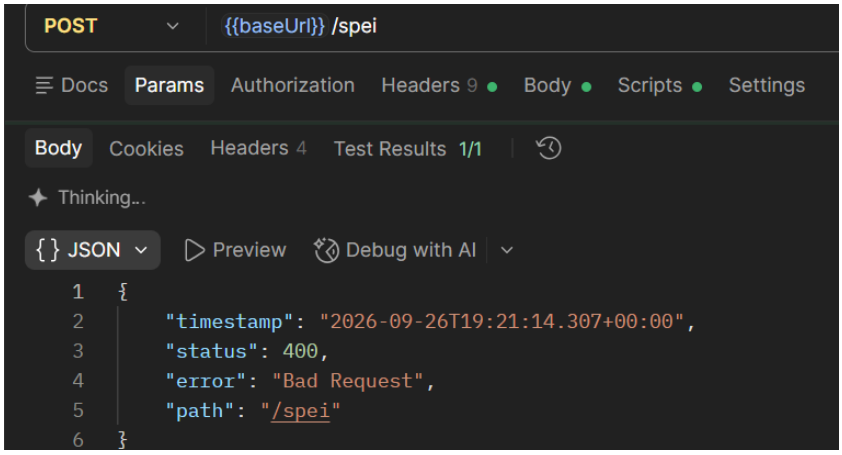
4. Cuando hay saldo insuficiente



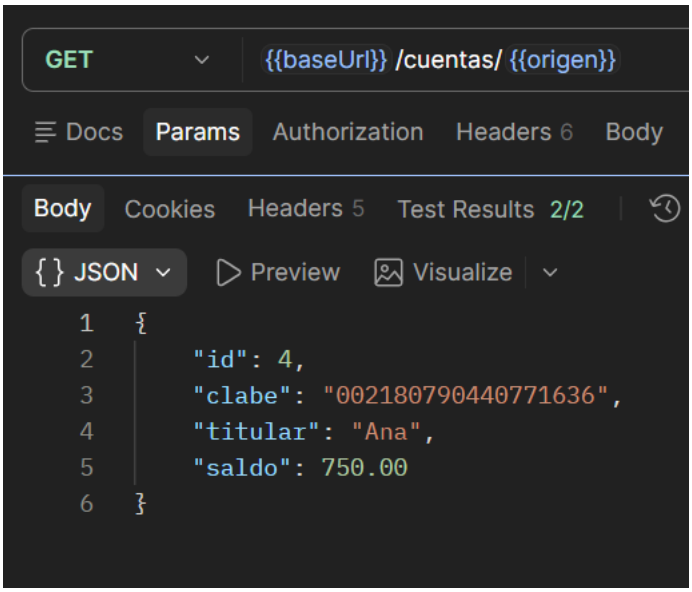
5. Cuando la cuenta destino y origen s la misma



6. SPEI sin idempotencia.

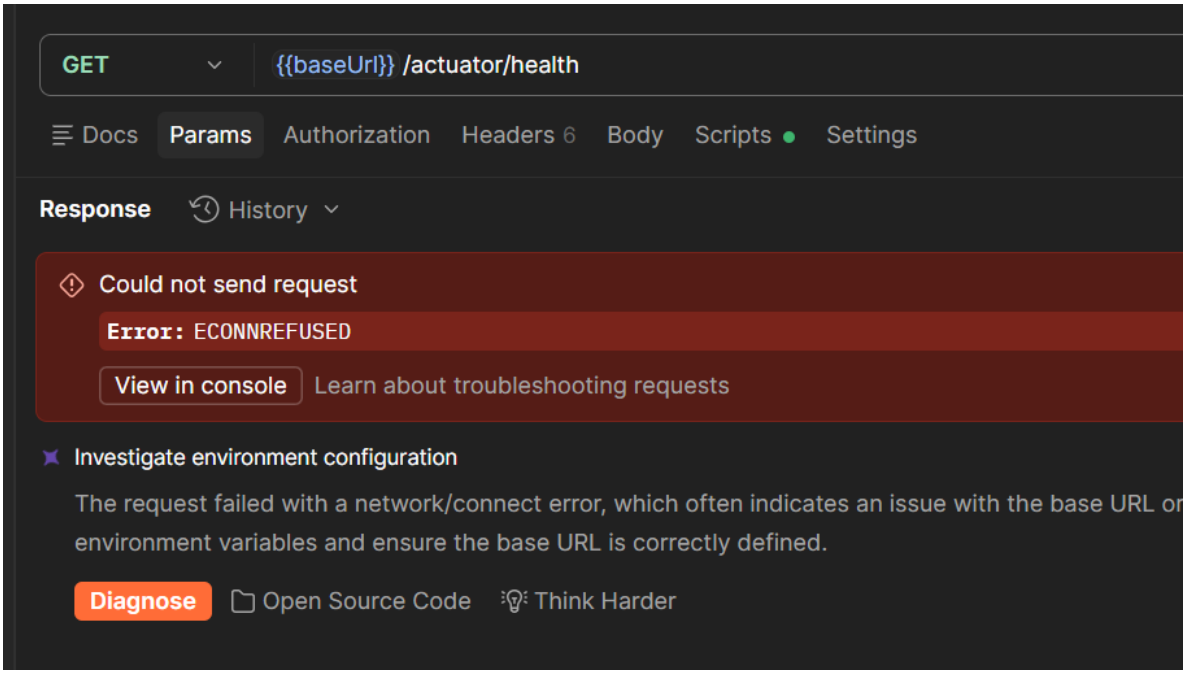


Hay que notar que a ser errores anteriores no hubo cambios, lo cual mantiene consistente al sistema.

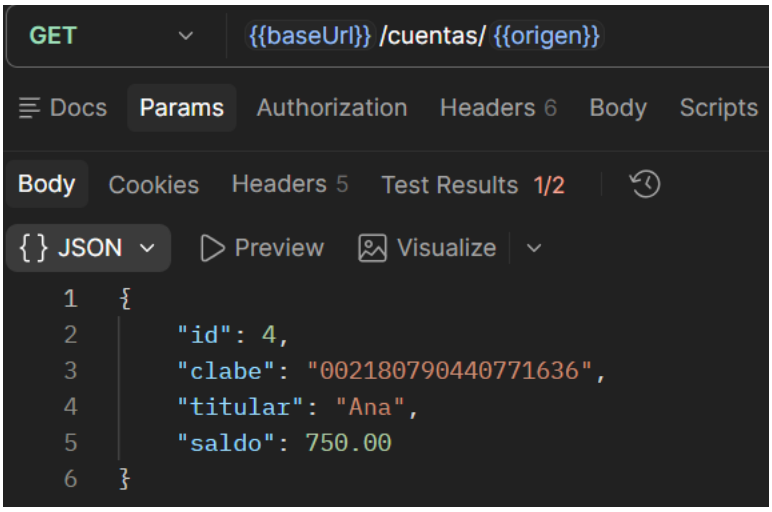


Paso 4. Bájalo con y sin volumen

Cuando Docker Compose down, podemos checar el salud desde Postman y podemos observar que no hay conexión con el localhost.

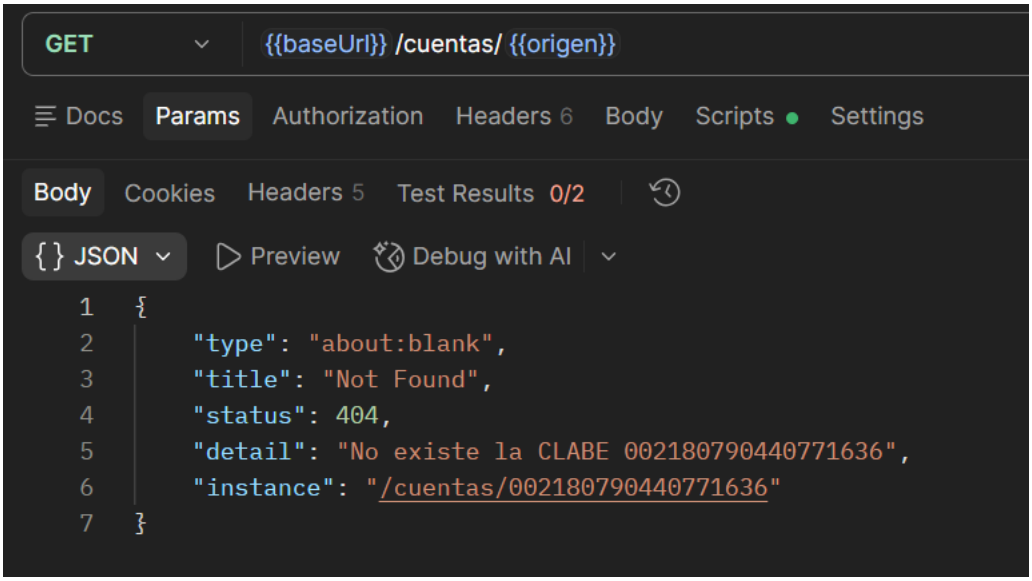


Posteriormente con Docker Compose up, podemos consultar los mismo datos aun con al transacciones ya realizadas.

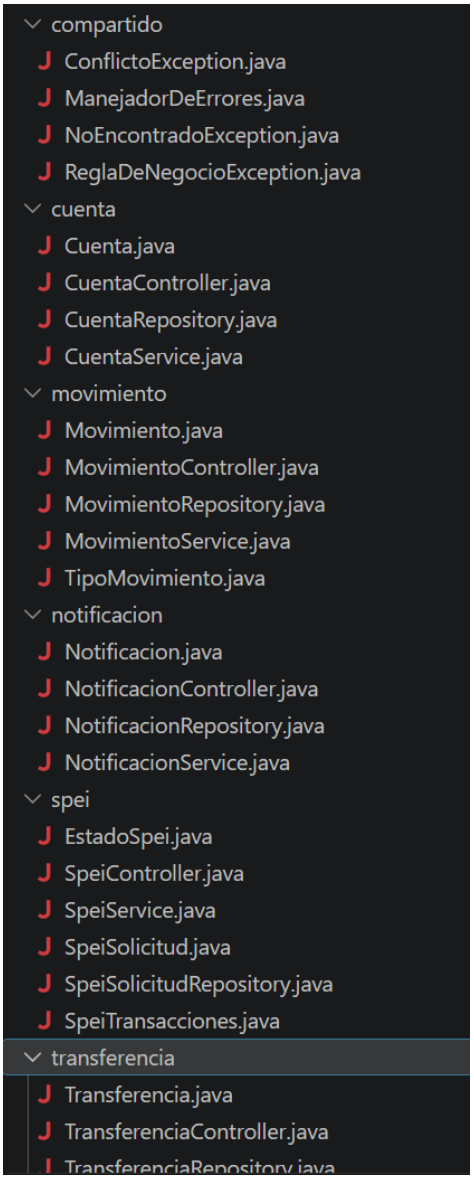


Si, eliminamos el contenedor y sus volumen, ocurre un problema a la información se pierde y no podemos consultarla, como aparece en error que no la CLABE no existe.

```
D:\mexi-banco>docker compose down -v
[+] down 4/4
 ✔Container mexi-banco-app-1          Removed
 ✔Container mexi-banco-db-1           Removed
 ✔Network mexibanco                   Removed
 ✔Volume mexi-banco_mexibanco_datos Removed
```



Paso 5. Mexi-Banco



- **Compartido:** Agrupa clases transversales y de uso general, como excepciones personalizadas y el manejo centralizado de errores.

- **Cuenta:** Contiene los datos y la lógica de las cuentas bancarias (entidad, controlador, repositorio y servicio).
- **Movimiento:** Administra los registros de transacciones financieras individuales mediante su entidad, controlador, repositorio, servicio y tipos de movimiento.
- **Notificacion:** Gestiona el envío de avisos y alertas del sistema con su respectiva estructura modular.
- **Spei:** Integra los componentes para procesar los pagos electrónicos, abarcando estados, solicitudes, transacciones, repositorio, servicio y controlador.
- **Transferencia:** Maneja las operaciones de transferencia de fondos entre cuentas a través de su entidad, controlador y repositorio.

- **Paso 6. Docker y Docker-compose**

```
# ---- ETAPA 1: CONSTRUCCION ----
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY mvnw .
COPY .mvn .mvn
RUN apk add dos2unix && dos2unix ./mvnw && chmod +x ./mvnw && ./mvnw dependency:go-offline -B
COPY src ./src
RUN ./mvnw clean package -DskipTests

# ---- ETAPA 2: EJECUCION ----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Este Dockerfile utiliza una construcción en dos etapas (multi-stage build) para empaquetar la aplicación de manera eficiente: en la primera etapa (builder), toma el código fuente, descarga las dependencias y compila el proyecto usando Maven con Java 21 para generar el archivo ejecutable (.jar); luego, en la segunda etapa, toma únicamente ese archivo empaquetado y lo coloca en una imagen ligera de Java optimizada para producción, exponiendo el puerto 8080 y configurando el comando de inicio para ejecutar la aplicación.

```yaml
services:
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: mexibanco
      POSTGRES_USER: mexibanco
      POSTGRES_PASSWORD: mexibanco
    volumes:
      - mexibanco_datos:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U mexibanco -d mexibanco"]
      interval: 5s
      timeout: 3s
      retries: 5

  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      DB_HOST: db
      DB_PORT: "5432"
      DB_NAME: mexibanco
      DB_USER: mexibanco
      DB_PASSWORD: mexibanco
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health | grep -q UP || exit 1"]
      interval: 10s
      timeout: 3s
      retries: 5

networks:
  default:
    name: mexibanco

volumes:
  mexibanco_datos:
```

El archivo comienza definiendo la sección services: para estructurar los dos contenedores principales del sistema que trabajarán en conjunto. Dentro de esta, el bloque db: descarga y levanta la base de datos PostgreSQL utilizando la imagen postgres:17-alpine, configurando sus credenciales de acceso en la sección environment: y asignando un espacio de almacenamiento seguro con volumes: - mexibanco_datos:/var/lib/postgresql/data para garantizar que la información persista aunque se apague el contenedor; asimismo, incluye un apartado de healthcheck: con el comando pg_isready para verificar constantemente que el motor de base de datos esté disponible y listo para recibir conexiones.

Por otro lado, el bloque app: utiliza la instrucción build: para compilar la aplicación a partir del código fuente y abre el acceso exterior mediante ports: - "8080:8080". En su

configuración de environment:, especifica los datos de conexión apuntando al servicio de base de datos (DB_HOST: db), vinculándose estrictamente mediante depends_on: con la condición service_healthy para asegurar que la aplicación no arranque hasta que PostgreSQL esté completamente operativo, y añade su propio healthcheck: consultando la ruta web /actuator/health.

Finalmente, las últimas secciones del archivo configuran networks: para aislar y conectar ambos servicios bajo una red privada común llamada mexibanco, facilitando su comunicación interna de manera segura, y cierran declarando el volumen mexibanco_datos para habilitar el almacenamiento persistente de la base de datos.